Project Design Phase

Smart Lender: AI-Powered Loan Approval Prediction System

System Architecture

The Smart Lender system is designed using a layered architecture to ensure efficient data processing, prediction, and user interaction. The architecture consists of four major layers:

Presentation Layer

- Provides a user-friendly web interface developed using **Flask, HTML, and CSS**.
- Allows users to enter applicant details such as income, education, employment status, credit history, and loan information.

Application Layer

- Receives user input from the web application.
- Validates the entered information.
- Performs preprocessing before sending the data to the Machine Learning model.

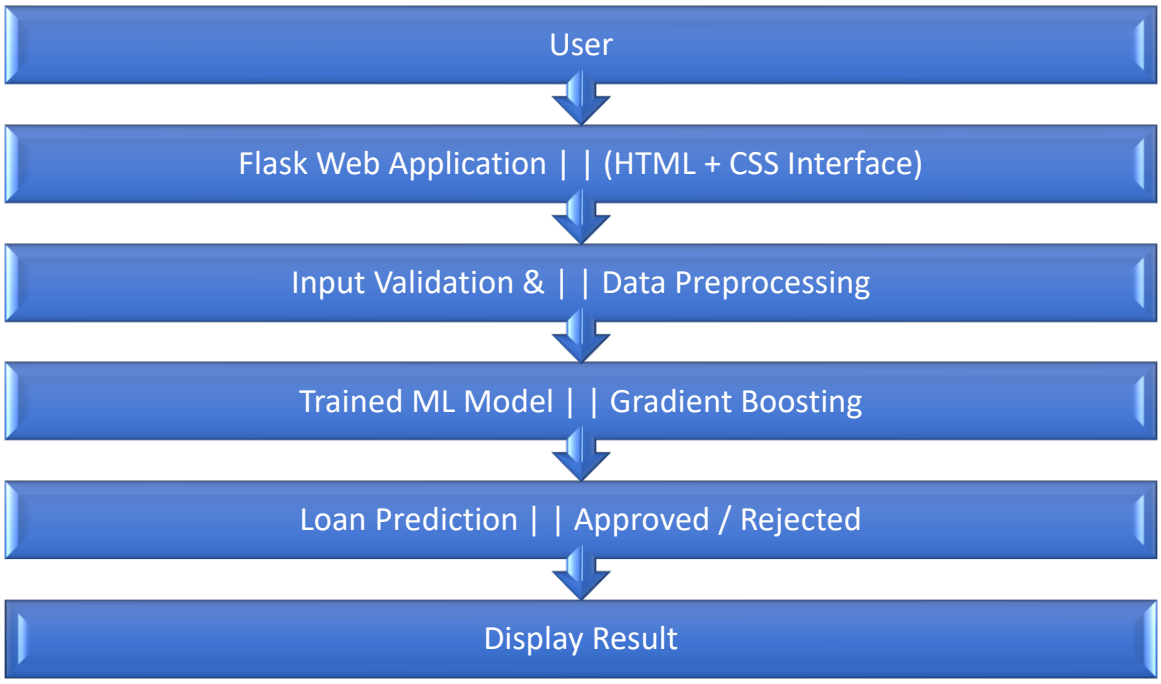
Machine Learning Layer

- Contains the trained **Gradient Boosting Classifier**.
- Uses preprocessing techniques including label encoding and feature scaling.
- Generates loan approval predictions.

Deployment Layer

- Runs locally using the Flask framework.
- Can be extended for cloud deployment in the future using platforms such as Microsoft Azure, AWS, or IBM Cloud.

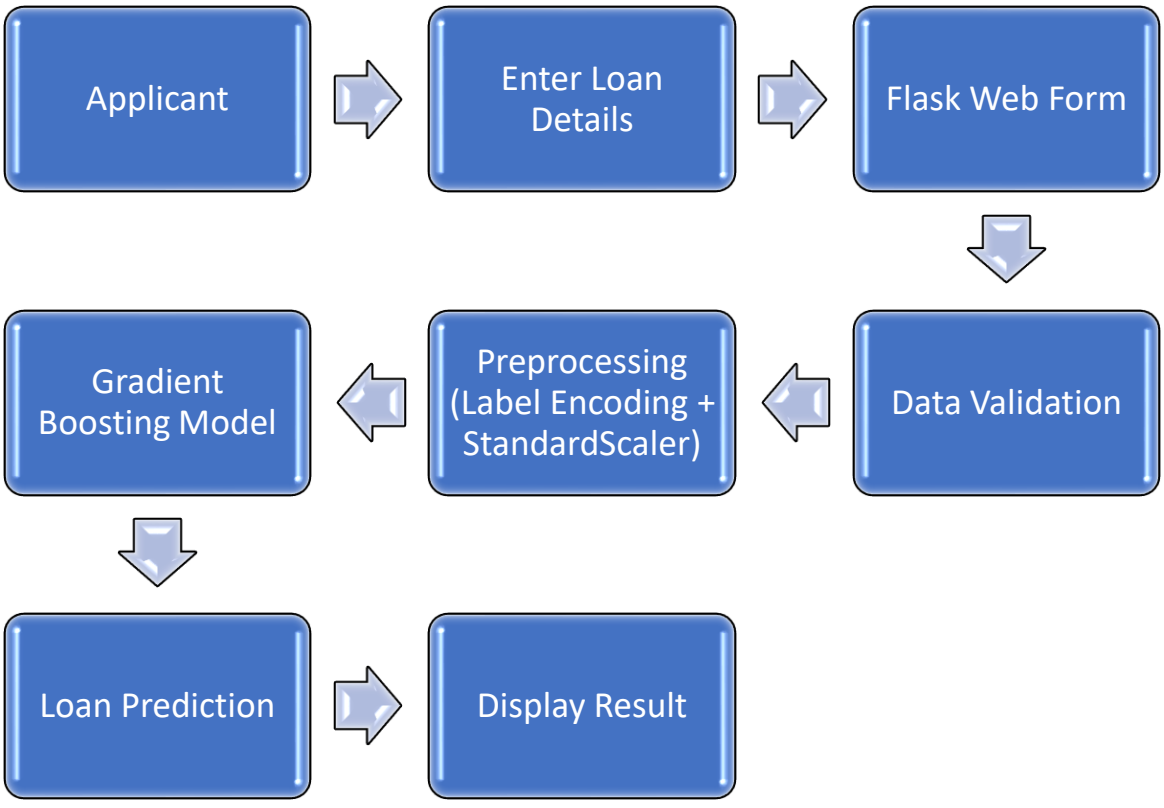
System Architecture Diagram



Data Flow

The Smart Lender application collects applicant information through the Flask web interface.
The input data is validated and preprocessed by handling categorical values and applying feature scaling using the same preprocessing techniques employed during model training.
The processed data is then passed to the trained Gradient Boosting model, which predicts whether the loan application is likely to be approved or rejected.
Finally, the prediction result is displayed to the user through the web application.

Data Flow Diagram (DFD)



Model Design

The Machine Learning model is developed through the following stages:

- Missing values are handled using appropriate imputation techniques.
- Categorical features are converted into numerical values using label encoding.

- Dataset imbalance is addressed using **SMOTE (Synthetic Minority Oversampling Technique)**.
- Feature scaling is performed using **StandardScaler**.
- The dataset is divided into training and testing sets.
- Multiple Machine Learning algorithms are trained and compared.

The models evaluated include:

- Decision Tree Classifier
- Random Forest Classifier
- K-Nearest Neighbors (KNN)
- Gradient Boosting Classifier

Among these models, the **Gradient Boosting Classifier** achieved the highest testing accuracy and was selected for deployment.

The trained model is saved using **Pickle**, allowing it to be loaded directly by the Flask application during prediction